

Human motion analysis

Rigotti Leonardo
University of Trento
leonardo.rigotti@studenti.unitn.it

July 15, 2025

Abstract

In the field of video surveillance and human behaviour analysis, the human motion detection and tracking plays an important role. In this project we implement a "pre Deep Learning" solution to do the tasks of motion detection and tracking. We will use a collection of video from the Stanford Drone Dataset that consists of UAV surveillance videos. This is a project report so it will include the main goals of the project and a subsection for each of them, consisting in a concise explanation of the State of the Art for that task, the solution(s) adopted and the results obtained.

1 Introduction

The main goal is to implement a solution using traditional Computer Vision methods, not exploiting Artificial Intelligence, to reach good results on the tasks prefixed. Referring to "pre Deep Learning" methods, we mean all those techniques that do not require to train a model (or to have a pre-trained one) to accomplish a task. Of course, the results obtained are different that if using AI models, but this approach allows you to implement a lightweight solution that tradeoffs goodness of results and computational power and efficiency. The dataset consists in video made from a Unmanned Aerial Vehicle, basically a drone, filming several parts of the campus of the Stanford University for surveillance purposes. The goal is to detect moving objects (without classifying them) and compute their trajectories.

2 Context and environment

The complete list of tasks, numbered in the order in which they were approached is:

1. Implement a detection and tracking system for the moving objects;
2. Compute the trajectories for the detected moving objects;

3. Compute an accuracy metric of the obtained trajectories, using for example the Mean Squared Error;
4. Considering all objects, based on the ground truth, what is the most frequent path? (e.g. objects enter left and exit top)

All the project was developed in a isolated Python environment, the tests were run locally and the following libraries were exploited:

- OpenCV [3]
- matplotlib [5]
- numpy [4]
- pandas [6]

We now start discussing each of them in a separated subsection, with an initial short digression on the dataset and the videos used. The discussion will be articulated in a brief presentation of the State of the Art for that task, a complete explanation of the techniques adopted, and a critic study of the results.

3 Dataset

The Stanford Drone Dataset, or SDD, was the first large scale dataset [9] (around 69 G) of videos made from above to a crowded scenario, in this case a university campus, including various types of agents (pedestrian, bikers, motor vehicles, etc.) that interact among each other. It was created by Stanford members (A. Robicquet et al.) to study interaction between the people crowding the streets in order to improve algorithms that study that type of trajectories [9].

In this project we used the "little" version (findable on Kaggle), which consists of four videos, in particular, we used only "video0" and "video3".

The videos are very similar, they include a roundabout and several pedestrians and bikers that cross it: see Figure 1. The background is always the same, since the drone stays still while filming. The luminance remains pretty the same, with some noise caused by the leaves moving for the wind. As you can see in Figure 1 there are a lot of shadows, both moving (pedestrians and bikers) and not (trees and vegetation).



Figure 1: A reference from video0

4 Detection and tracking

This task consists in detecting the moving objects, removing possible noise caused by shadows, change of luminance or wind, and compute their trajectories.

4.1 State of the Art

In modern computer vision, motion detection is often handled using deep learning-based object detection models. Techniques like YOLO (You Only Look Once) [7] and Faster R-CNN [8] are widely used to identify objects in each frame, and by comparing detections across frames, motion can be inferred. Unlike traditional background subtraction, these models do not rely on a static background and are more robust to environmental noise, illumination changes, and dynamic scenes.

All modern algorithms are based on the deep learning mechanism of Convolutional Neural Networks. The base idea of CNN is to detect features from an image, or a portion of an image, applying several convolution filters at different layers. As the image passes through the network, the space dimensions are reduced. Final scores for different features are eventually used to classify the image. The strength of such networks is their ability to learn the features directly from data.

4.2 Methods applied

It should be pointed out that both videos are very similar in terms of content. The main difference between the two is the durations and, clearly, the number of actors that appears. Based on that, the solution applied to one video can be easily applied also to the other,

expecting the same goodness of results.

The background remains the same for all the duration, since the drone is still. This allows to apply the technique of background subtraction. Background subtraction is a technique to generate a foreground mask, a binary image containing white pixels in correspondence of moving objects. If an object is detected moving, it gets assigned to the foreground and gets white pixel, viceversa it goes in the background and gets black [3].

This feature is in-built in cv2 library, that offers various methods to do background subtraction. The most valuable for this case, found empirically exploring through available algorithms, was found in KNN (K-Nearest-Neighbors) subtractor. This algorithm keeps for each pixel an history of recent samples, a new value for the pixel is then compared to this history and if it is within a threshold from K of them, it gets assigned to the background, otherwise to the foreground. It is an adaptive algorithm, so objects that remain static long enough get assigned to background. [11] In OpenCV library it is a parametric algorithm, so you can configure N, the number of data points in the history and K, the number of neighbors. There is also the flag `detectShadows`, that is set to `True` should detect and remove shadows, but as you can see in Figure 2 is weak in our use case. To overcome the problem of shadows, a smoothing filter is applied to the original frame. In this case it is a Gaussian Blur with a kernel of 25x25. This measure for the kernel was found by trying several alternatives, starting from 5x5, and selecting the best one.

After generating the motion mask frame, a dilation ellipse shaped of 3x3 kernel is applied to fulfill generated by background subtraction, in order to get smooth a dense object, easier to be recognized by the contours detector.



(a) 500th frame



(b) 500th motion mask

Figure 2: Comparison between 500th frame and motion mask from video3

5 Motion tracking

The motion tracking task consists of computing trajectories for each moving object in the scenario by analyzing spatial and temporal changes.

5.1 State of the Art

An interesting framework to do the motion tracking task is Deep SORT [10], that combines SORT algorithm with appearance information, improving the performance of SORT in presence of long occlusions, reducing significantly the amount of identity switches.

5.2 Methods applied

The motion tracking task is performed using the Simple Online and Realtime Tracking (SORT) algorithm [1]. This lightweight and efficient tracking framework is designed for real-time applications and provides robust performance when paired with reliable object detections. In this implementation, moving objects are first extracted by detecting contours using the `findContours` function from OpenCV, which identifies object boundaries based on motion masks. Once the moving objects are detected in each frame, their bounding boxes are passed to the SORT algorithm for tracking. Before passing the bounding box, they detections with an area less than 500 pixels get discarded.

SORT works by maintaining a set of object trajectories and updating them frame by frame. It relies on a combination of a Kalman filter for motion prediction and the Hungarian algorithm for data association. The Kalman filter predicts the future positions of objects based on their previous motion patterns. The Hungarian algorithm is then used to associate new detections with existing tracks by minimizing the distance between predicted and observed positions. [2]

To have a visual feedback of the tracking, a rectangle is drawn around the detected object, with a label that indicates its ID in the tracker.

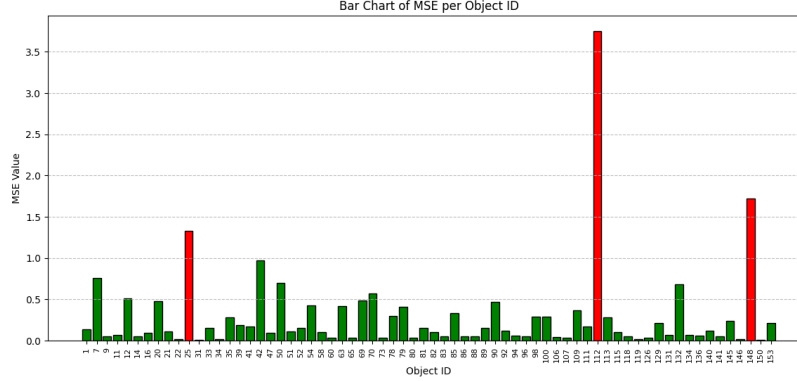


Figure 4: Histogram of Mean Squared Error per Object ID, red bars mean MSE greater than 1

7 Most frequent path

A dedicated script analyzes the object trajectories from the annotation file present in the dataset to determine where each object enters and exits the frame. A reference image is used to determine the dimensions of the video frame, which are essential for identifying whether an object appears near the left, right, top, bottom, or center of the frame.

For each object, the first and last recorded positions are examined to estimate the entry and exit sides. In cases where these positions lie too close to the center an approximation is used. Specifically, the trajectory is treated as linear, and its direction is extrapolated based on nearby points.

An object entry/exit point is assigned to a side if its first/last coordinate is within a margin of the 20% of the frame. For objects entering/exiting in the corners (within 20% of two sides), the order of assign is left, right, top, bottom (e.g. an object enters from the left-bottom corner, it gets assigned left).

The pairs of entry and exit points are stored in a dictionary which is then sorted to get the most frequent path.

8 Results

The results consist in:

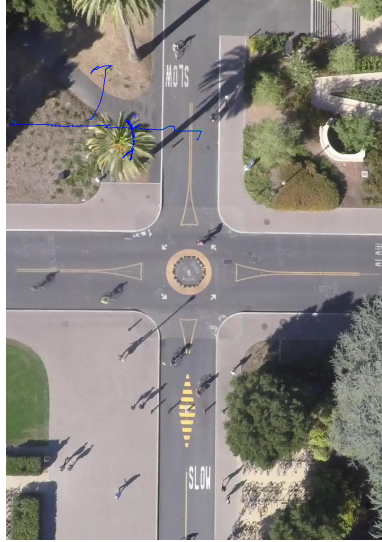
- The full sequence of frames (video) containing tagged bounding boxes
- The MSE histogram
- An image representing the outlier trajectories (i.e. the ones with a MSE greater than 1)

- The result of the most frequent path analysis

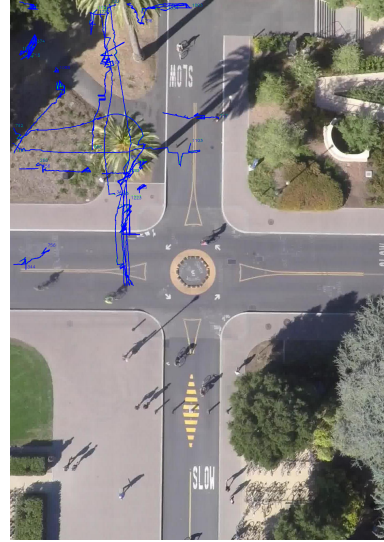
It can be seen that the detections often merge when two moving objects get too close to each other. This is inescapable using a method like Background Subtraction to do the motion mask. Anyway, when two objects then separate, the tracker re-assign them new different IDs. Because of that, some trajectories can be split, but that does not interfere with the computation of MSE. On the other hand this method cannot be applied to study interaction between moving object, since they become a single object when approaching each other.

Overall, this is a very lightweight and good solution to detect moving objects to perform tasks such as the most frequent path estimation.

What concerning MSE, the trajectories with a result greater than 1 were considered as "outliers" and they were drawn in a separate frame, with the reference image as background. As you can see in Figure 5, those trajectories have a lot of spikes and sudden change of direction, maybe due to a merging with other near blobs. In addition, we can see that all the outliers are in the top-left corner, this is due maybe to the high noise generated by the plants moving there. In fact the plants and shadows problem is quite fully resolved with blurring, minimum area for detections and filtering for short trajectories, but can still disturb the tracking algorithm.



(a) Outliers for video0



(b) Outliers for video3

Figure 5: Comparison between outliers for video0 and video3

All the other material generated by the scripts can be found in the delivery folder, since it's not practical to insert them here.

The result for most frequent path analysis:

From	To	Count
left	right	9
bottom	right	9
right	left	6
left	bottom	6
left	left	5
bottom	top	5
top	left	2
right	right	2
bottom	left	2
top	bottom	2
right	bottom	2
right	top	1

Table 1: Most frequent path for video0

From	To	Count
right	left	46
left	right	40
bottom	left	33
bottom	top	23
bottom	right	21
right	bottom	17
top	bottom	15
left	bottom	14
top	right	11
right	right	8
top	left	7
bottom	bottom	7
left	top	7
left	left	4
right	top	3
top	top	1

Table 2: Most frequent path for video3

From these results, it can be deduced that the most frequent path for video0 is **left** $\rightarrow$ **right**, while for video3 it is **right** $\rightarrow$ **left**. The objects entering and exiting from the same side can be intended as errors, because of some sudden changes in the trajectory due to noise, the linearization approximation could "send back" the trajectory to its starting point, or, as it can be see in the video, some people entering from the green spots (so not on the road) and then reach the road.

9 Conclusion

This project tried a lightweight legacy approach to Computer Vision's tasks like motion tracking and detection, reaching fairly good results for a system of surveillance or statistical calculations. To improve the system, a part of introducing Artificial Intelligence in pretty all the aspects of the project, an algorithm that merge split trajectories could be implemented, so that objects that merge and then split would have only one right trajectory.

References

- [1] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.

- [2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, September 2016.
- [3] G. Bradski. The OpenCV Library. *Dr. Dobb’s Journal of Software Tools*, 2000.
- [4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [5] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [6] The pandas development team. pandas-dev/pandas: Pandas, February 2020.
- [7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016.
- [8] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks, 2016.
- [9] Alexandre Robicquet, Amir Sadeghian, Alexandre Alahi, and Silvio Savarese. Learning social etiquette: Human trajectory prediction in crowded scenes. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016.
- [10] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric, 2017.
- [11] Zoran Zivkovic and Ferdinand van der Heijden. Efficient adaptive density estimation per image pixel for the task of background subtraction. *Pattern Recognition Letters*, 27(7):773–780, 2006.